



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

**Faculty of
Computing**

SECP3623

Database Programming

2025/2026 SEMESTER 1

Assignment 1 (10%)

SECTION : 01

LECTURER : DR. ROZILAWATI BINTI DOLLAH @ MD. ZAIN

NABIL AFLAH BOO BINTI MOHD YOSUF BOO YONG CHONG	A23CS0252
AFIF SHAQIR IRFAN BIN ARQAM	A23CS0204

- **TASK 1 - Database Creation and Integrity Constraints (CLO1)**

```
DROP DATABASE IF EXISTS hostel_mgmt_aflah_afif; #check if database exists
CREATE DATABASE hostel_mgmt_aflah_afif;
USE hostel_mgmt_aflah_afif;
```

Table for Room Types

```
#TASK 1:
#create table
CREATE TABLE room_types (
    type_id INT AUTO_INCREMENT PRIMARY KEY,
    type_name VARCHAR(50) UNIQUE NOT NULL,
    rent DECIMAL(10,2) NOT NULL,
    deposit DECIMAL(10,2) NOT NULL,
    capacity INT NOT NULL
);
```

Table for Rooms

```
CREATE TABLE rooms (
    room_id INT AUTO_INCREMENT PRIMARY KEY,
    type_id INT NOT NULL,
    room_no VARCHAR(10) UNIQUE NOT NULL,
    floor_no INT NOT NULL,
    is_occupied BOOLEAN DEFAULT FALSE, #true=occupied, false=vacant
    FOREIGN KEY (type_id) REFERENCES room_types(type_id)
        ON UPDATE RESTRICT
        ON DELETE CASCADE
);
```

Table for Students

```
CREATE TABLE students (  
    student_id INT AUTO_INCREMENT PRIMARY KEY,  
    room_id INT NOT NULL,  
    fname VARCHAR(100) NOT NULL,  
    lname VARCHAR(100) NOT NULL,  
    status ENUM('ACTIVE', 'NON_ACTIVE') NOT NULL,  
    checkin_date DATE NOT NULL,  
    FOREIGN KEY (room_id) REFERENCES rooms(room_id)  
        ON UPDATE RESTRICT  
        ON DELETE CASCADE  
);
```

Table for maintenance

```
CREATE TABLE maintenance (  
    maint_id INT AUTO_INCREMENT PRIMARY KEY,  
    room_id INT NOT NULL,  
    issue_desc TEXT NOT NULL,  
    severity ENUM('LOW', 'MEDIUM', 'HIGH') NOT NULL,  
    status ENUM('OPEN', 'RESOLVED') NOT NULL,  
    reported_on DATE,  
    resolved_on DATE,  
    FOREIGN KEY (room_id) REFERENCES rooms(room_id)  
        ON UPDATE RESTRICT  
        ON DELETE CASCADE  
);
```

Table for payments

```
CREATE TABLE payments (  
    payment_id INT AUTO_INCREMENT PRIMARY KEY,  
    student_id INT NOT NULL,  
    amount DECIMAL(10,2) NOT NULL,  
    paid_on DATE NOT NULL,  
    method ENUM('CASH', 'FPX', 'CARD', 'TNG') NOT NULL,  
    note TEXT,  
    FOREIGN KEY (student_id) REFERENCES students(student_id)  
        ON UPDATE RESTRICT  
        ON DELETE CASCADE  
);
```

Demonstrate schema maintenance:

- ❖ ALTER TABLE: add an email column to students and mark it UNIQUE.
- ❖ DROP TABLE: create a temporary test table and drop it safely.

```
ALTER TABLE students  
ADD email VARCHAR(100) UNIQUE;
```

```
CREATE TABLE test (  
    test_id INT PRIMARY KEY,  
    description TEXT  
);  
DROP TABLE test;
```

- **TASK 2 - Data Manipulation and Filtering (CLO2)**

1. Data Insertion (DML): Insert at least 10 rows per table with realistic and varied data as follows:

room_types: Include multiple rent levels, with at least one rent value between RM400 and RM800, and ensure that deposits and capacities vary meaningfully.

#TASK 2:

```
INSERT INTO room_types(type_name, rent, deposit, capacity) VALUES
  ('Single', 300.00, 150.00, 1),
  ('Double', 400.00, 200.00, 2),
  ('Premium', 800.00, 350.00, 2),
  ('Family', 1000.00, 425.00, 4),
  ('Triple', 600.00, 300.00, 3),
  ('Dorm', 750.00, 375.00, 6),
  ('Executive', 1500.00, 750.00, 3),
  ('Suite', 1200.00, 600.00, 2),
  ('Deluxe', 550.00, 275.00, 2),
  ('Private', 350.00, 175.00, 1);
```

rooms: Distribute rooms across a minimum of three floors, ensuring that each room is correctly linked to a valid room type.

```
INSERT INTO rooms(type_id, room_no, floor_no, is_occupied) VALUES
(1, 'A101', 1, FALSE),
(2, 'B102', 1, FALSE),
(3, 'C201', 2, FALSE),
(4, 'D301', 3, FALSE),
(5, 'E201', 2, FALSE),
(6, 'F101', 1, FALSE),
(7, 'G301', 3, FALSE),
(8, 'H201', 2, FALSE),
(9, 'I201', 2, FALSE),
(10, 'J301', 3, FALSE);
```

students: Include both 'ACTIVE' and 'NON_ACTIVE' students. At least one student name must begin with the letter 'A', and every ACTIVE student must be assigned to an existing room.

```
INSERT INTO students(room_id, fname, lname, status, checkin_date, email) VALUES
(1, 'Siti', 'Alia', 'ACTIVE', '2025-06-12', 'sitialia@graduate.utm.my'),
(2, 'Ahmad', 'Ali', 'NON_ACTIVE', '2025-07-13', 'ahmadali@graduate.utm.my'),
(3, 'Abu', 'Bakar', 'ACTIVE', '2025-08-14', 'abubakar@graduate.utm.my'),
(4, 'Farah', 'Zahid', 'ACTIVE', '2025-09-15', 'farahzahid@graduate.utm.my'),
(5, 'Lin', 'Yi', 'NON_ACTIVE', '2025-06-16', 'linyi@graduate.utm.my'),
(6, 'Xin', 'Ya', 'ACTIVE', '2025-07-17', 'xinya@graduate.utm.my'),
(7, 'Ahmad', 'Zaki', 'NON_ACTIVE', '2025-10-26', 'ahmad_z@graduate.utm.my'),
(8, 'Nisa', 'Sofia', 'ACTIVE', '2025-10-13', 'nisasofia@graduate.utm.my'),
(9, 'Daniel', 'Lee', 'NON_ACTIVE', '2025-09-27', 'daniel_lee@graduate.utm.my'),
(10, 'Saida', 'Nur', 'ACTIVE', '2025-08-17', 'saidanur@graduate.utm.my');
```

maintenance: Include diverse records with different severity levels (LOW, MEDIUM, HIGH) and statuses (OPEN, RESOLVED).

```
INSERT INTO maintenance(room_id, issue_desc, severity, status, reported_on, resolved_on) VALUES
(1, 'Tap is leaking', 'LOW', 'OPEN', '2025-10-02', NULL),
(2, 'Toilet cannot flush', 'MEDIUM', 'OPEN', '2025-06-13', NULL),
(3, 'Ceiling fan is slow', 'LOW', 'OPEN', '2025-10-17', NULL),
(4, 'Door knob loose', 'LOW', 'RESOLVED', '2025-07-14', '2025-07-21'),
(5, 'Power socket fault', 'HIGH', 'RESOLVED', '2025-09-24', '2025-09-25'),
(6, 'Window jammed', 'HIGH', 'OPEN', '2025-10-13', NULL),
(7, 'Aircond not working', 'HIGH', 'OPEN', '2025-09-23', NULL),
(8, 'Toilet door cannot lock', 'MEDIUM', 'RESOLVED', '2025-07-04', '2025-07-14'),
(9, 'Broken bed frame', 'MEDIUM', 'OPEN', '2025-10-15', NULL),
(10, 'Lamp is not working', 'HIGH', 'RESOLVED', '2025-08-25', '2025-08-27');
```

payments: Include payment records covering at least two different months and using at least two different payment methods among {CASH, FPX, CARD, TNG}. Every payment must be linked to an existing student.

```
INSERT INTO payments(student_id, amount, paid_on, method, note) VALUES
(1, 300.00, '2025-09-18', 'FPX', 'September rent'),
(2, 400.00, '2025-07-23', 'CARD', 'July rent'),
(3, 800.00, '2025-06-12', 'FPX', 'June rent'),
(4, 1000.00, '2025-08-18', 'CARD', 'Aug rent'),
(5, 600.00, '2025-09-23', 'TNG', 'Sep rent'),
(6, 750.00, '2025-10-11', 'CARD', 'Oct rent'),
(7, 1500.00, '2025-10-23', 'CARD', 'Oct rent'),
(8, 1200.00, '2025-06-09', 'CARD', 'June rent'),
(9, 550.00, '2025-08-13', 'FPX', 'Aug rent'),
(10, 350.00, '2025-07-16', 'CASH', 'Jul rent');
```

2. UPDATE and DELETE Operations:

- Update the is_occupied column in rooms to TRUE for rooms with ACTIVE students; otherwise, set it to FALSE.
- Delete maintenance records where status = 'RESOLVED' and the reported date is older than 60 days (if applicable).

```
#update is_occupied
```

```
SET SQL_SAFE_UPDATES = 0;
```

```
UPDATE rooms SET is_occupied = FALSE;
```

```
UPDATE rooms r
```

```
JOIN(
```

```
    SELECT DISTINCT room_id FROM students WHERE status='ACTIVE'
```

```
) active_rooms
```

```
ON r.room_id = active_rooms.room_id
```

```
SET r.is_occupied = TRUE;
```

```
#delete maintenance records (status='RESOLVED' & reported date >60 days)
```

```
DELETE FROM maintenance
```

```
WHERE status = 'RESOLVED'
```

```
AND reported_on < DATE_SUB(CURDATE(), INTERVAL 60 DAY);
```


3. Data Retrieval and Filtering Queries:
Write one query for each of the following:

- BETWEEN: Display room types where rent is between RM400 and RM800.

#data retrieval & filtering queries

```
SELECT *  
FROM room_types  
WHERE rent BETWEEN 400 AND 800;
```

OUTPUT:

	type_id	type_name	rent	deposit	capacity
▶	2	Double	400.00	200.00	2
	3	Premium	800.00	350.00	2
	5	Triple	600.00	300.00	3
	6	Dorm	750.00	375.00	6
	9	Deluxe	550.00	275.00	2
•	NULL	NULL	NULL	NULL	NULL

- LIKE: Retrieve students whose names (fname) start with the letter 'A'.

```
SELECT student_id, fname, lname, email  
FROM students  
WHERE fname LIKE 'A%';
```

OUTPUT:

	student_id	fname	lname	email
▶	2	Ahmad	Ali	ahmadali@graduate.utm.my
	3	Abu	Bakar	abubakar@graduate.utm.my
	7	Ahmad	Zaki	ahmad_z@graduate.utm.my
•	NULL	NULL	NULL	NULL

- IN: Display payments where the payment method is IN ('FPX', 'CARD').

```
SELECT *
FROM payments
WHERE method IN ('FPX', 'CARD');
```

OUTPUT:

	payment_id	student_id	amount	paid_on	method	note
▶	1	1	300.00	2025-09-18	FPX	September rent
	2	2	400.00	2025-07-23	CARD	July rent
	3	3	800.00	2025-06-12	FPX	June rent
	4	4	1000.00	2025-08-18	CARD	Aug rent
	6	6	750.00	2025-10-11	CARD	Oct rent
	7	7	1500.00	2025-10-23	CARD	Oct rent
	8	8	1200.00	2025-06-09	CARD	June rent
	9	9	550.00	2025-08-13	FPX	Aug rent
•	NULL	NULL	NULL	NULL	NULL	NULL

- Combine AND, OR, and NOT in at least one logical filter query.

#Combine AND, OR, and NOT in at least one logical filter query.

```
SELECT *
FROM students
WHERE
    (status = 'ACTIVE' AND fname LIKE 'A%')
    OR
    (NOT email LIKE '%utm.my');
```

OUTPUT :

[illegible]

4. Functions and Expressions:

Use SQL functions as follows (no duplication of function types):

- Aggregate Function: Use at least one among COUNT(), SUM(), AVG(), MIN(), or MAX().

```
#functions & expression
SELECT COUNT(*) AS total_active_students
FROM students
WHERE status = 'ACTIVE';
```

OUTPUT :

	total_active_students
▶	6

- String Functions: Use any two among UPPER(), LOWER(), LENGTH(), or CONCAT() in your queries.

```
SELECT UPPER(CONCAT(fname, ' ', lname)) AS fullname_upper #uppercase for all names
FROM students;
```

OUTPUT:

	fullname_upper
▶	SITI ALIA
	AHMAD ALI
	ABU BAKAR
	FARAH ZAHID
	LIN YI
	XIN YA
	AHMAD ZAKI
	NISA SOFIA
	DANIEL LEE
	SAIDA NUR

```
SELECT fname, lname, LENGTH(email) AS email_length #count the email length
FROM students;
```

OUTPUT :

	fname	lname	email_length
►	Siti	Alia	24
	Ahmad	Ali	24
	Abu	Bakar	24
	Farah	Zahid	26
	Lin	Yi	21
	Xin	Ya	21
	Ahmad	Zaki	23
	Nisa	Sofia	25
	Daniel	Lee	26
	Saida	Nur	24

- **Task 3 – Reporting and Aggregation (CLO2)**

1. Create a view v_room_status with columns:

(room_no, type_name, rent, floor_no, capacity, n_occupants, pending_issues, is_vacant)

- n_occupants = count of students with status='ACTIVE' per room.
- pending_issues = count of maintenance rows with status='OPEN' per room.
- is_vacant = TRUE when n_occupants = 0.

```
CREATE OR REPLACE VIEW v_room_status AS
```

```
SELECT
```

```
r.room_no,
```

```
rt.type_name,
```

```
rt.rent,
```

```
r.floor_no,
```

```
rt.capacity,
```

```
COUNT(DISTINCT CASE WHEN s.status = 'ACTIVE' THEN s.student_id END) AS n_occupants,
```

```
COUNT(DISTINCT CASE WHEN m.status = 'OPEN' THEN m.maint_id END) AS pending_issues,
```

CASE

```
    WHEN COUNT(DISTINCT CASE WHEN s.status = 'ACTIVE' THEN s.student_id END)=0
      THEN TRUE
    ELSE FALSE
  END AS is_vacant
```

FROM rooms r

JOIN room_types rt ON r.type_id = rt.type_id

LEFT JOIN students s ON r.room_id = s.room_id

LEFT JOIN maintenance m ON r.room_id = m.room_id

GROUP BY r.room_no, rt.type_name, rt.rent, r.floor_no, rt.capacity;

2. Summary queries (for the ≥ 10 -per-table dataset):

- Total number of students per room type.

#Total number of students per room type.

```
SELECT rt.type_name, COUNT(s.student_id) AS total_students
```

```
FROM room_types rt
```

```
JOIN rooms r ON rt.type_id = r.type_id
```

```
LEFT JOIN students s ON r.room_id = s.room_id
```

```
GROUP BY rt.type_name;
```

OUTPUT:

	type_name	total_students
►	Deluxe	1
	Dorm	1
	Double	1
	Executive	1
	Family	1
	Premium	1
	Private	1
	Single	1
	Suite	1
	Triple	1

- Average rent and total deposit per room type (use ROUND() to 2 d.p.).

```
#Average rent and total deposit per room type (use ROUND() to 2 d.p.).
SELECT type_name,
ROUND(AVG(rent), 2) AS average_rent,
ROUND(SUM(deposit), 2) AS total_deposit
FROM room_types
GROUP BY type_name;
```

OUTPUT:

	type_name	average_rent	total_deposit
▶	Deluxe	550.00	275.00
	Dorm	750.00	375.00
	Double	400.00	200.00
	Executive	1500.00	750.00
	Family	1000.00	425.00
	Premium	800.00	350.00
	Private	350.00	175.00
	Single	300.00	150.00
	Suite	1200.00	600.00
	Triple	600.00	300.00

- Monthly payment totals grouped by year & month (ensure your dataset spans ≥2 months).

```
#Monthly payment totals grouped by year & month (ensure your dataset spans ≥2 months).
SELECT
YEAR(paid_on) AS year_paid,
MONTH(paid_on) AS month_paid,
SUM(amount) AS total_paid

FROM payments
GROUP BY YEAR(paid_on), MONTH(paid_on)
ORDER BY year_paid, month_paid;
```

OUTPUT:

	year_paid	month_paid	total_paid
▶	2025	6	2000.00
	2025	7	750.00
	2025	8	1550.00
	2025	9	900.00
	2025	10	2250.00

- Count of OPEN maintenance issues per floor using HAVING COUNT > 2.

#Count of OPEN maintenance issues per floor using HAVING COUNT >2.

```
SELECT r.floor_no,  
COUNT(m.maint_id) AS issues  
FROM rooms r  
JOIN maintenance m ON r.room_id = m.room_id  
WHERE m.status = 'OPEN'  
GROUP BY r.floor_no  
HAVING COUNT(m.maint_id) >2;
```

OUTPUT :

	floor_no	issues
▶	1	3

3. Apply SQL functions in your report queries:

- ROUND() o UPPER()/LOWER()
- CONCAT()
- CASE to categorize room type rent as LOW / MEDIUM / HIGH (state your thresholds explicitly)

#Apply SQL functions in your report queries:

```
SELECT rt.type_name,
```

```
UPPER(rt.type_name) AS type_upper,
```

```
LOWER(rt.type_name) AS type_lower,
```

```
ROUND(rt.rent,2) AS rounded_rent,
```

```
CONCAT(rt.type_name, ' room costs RM ',rt.rent) AS rent_desc,
```

› CASE

```
WHEN rt.rent < 500 THEN 'LOW'
```

```
WHEN rt.rent BETWEEN 500 and 900 THEN 'MEDIUM'
```

```
ELSE 'HIGH'
```

↳ END AS rent_category

↳ FROM room_types rt;

OUTPUT :

	type_name	type_upper	type_lower	rounded_rent	rent_desc	rent_category
►	Single	SINGLE	single	300.00	Single room costs RM 300.00	LOW
	Double	DOUBLE	double	400.00	Double room costs RM 400.00	LOW
	Premium	PREMIUM	premium	800.00	Premium room costs RM 800.00	MEDIUM
	Family	FAMILY	family	1000.00	Family room costs RM 1000.00	HIGH
	Triple	TRIPLE	triple	600.00	Triple room costs RM 600.00	MEDIUM
	Dorm	DORM	dorm	750.00	Dorm room costs RM 750.00	MEDIUM
	Executive	EXECUTIVE	executive	1500.00	Executive room costs RM 1500.00	HIGH
	Suite	SUITE	suite	1200.00	Suite room costs RM 1200.00	HIGH
	Deluxe	DELUXE	deluxe	550.00	Deluxe room costs RM 550.00	MEDIUM
	Private	PRIVATE	private	350.00	Private room costs RM 350.00	LOW